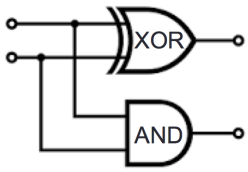
# Half adder circuit

**Fixed-width addition**: adding one bit at time, using the usual column-by-column and carry arithmetic, and dropping the carry from the leftmost column so the result is the same width as the summands. In many cases, this gives representation of the correct value for the sum when we interpret the summands in fixed-width binary or in 2s complement.

**Addition**: For single column:

| Input | | Output | |
|---|---|---|---|
| $x_0$ | $y_0$ | $c_0$ | $s_0$ |
| 1 | 1 | | |
| 1 | 0 | | |
| 0 | 1 | | |
| 0 | 0 | | |




CC BY-NC-SA 2.0 Version October 21, 2026 (1)

# Two bit adder circuit

Draw a logic circuit that implements binary addition of two numbers that are each represented in fixed-width binary:

- Inputs $x_0, y_0, x_1, y_1$ represent $(x_1x_0)_{2,2}$ and $(y_1y_0)_{2,2}$
- Outputs $z_0, z_1, z_2$ represent $(z_2z_1z_0)_{2,3} = (x_1x_0)_{2,2} + (y_1y_0)_{2,2}$ (may require up to width 3)

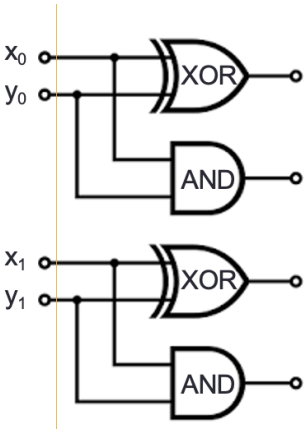
*First approach*: half-adder for each column, then combine carry from right column with sum of left column

Write expressions for the circuit output values in terms of input values:

$z_0 =$ _____

$z_1 =$ _____

$z_2 =$ _____




*There are other approaches, for example*: for middle column, first add carry from right column to $x_1$, then add result to $y_1$

Extra practice: how does this circuit change if we're representing fixed-width addition?

# Why represent numbers

Modeling uses data-types that are encoded in a computer. The details of the encoding impact the efficiency of algorithms we use to understand the systems we are modeling and the impacts of these algorithms on the people using the systems. Case study: how to encode numbers?

CC BY-NC-SA 2.0 Version October 21, 2026 (2)

# Base expansion definition

**Definition** For $b$ an integer greater than 1 and $n$ a positive integer, the **base $b$ expansion of $n$** is

$$(a_{k-1} \cdots a_1 a_0)_b$$

where $k$ is a positive integer, $a_0, a_1, \ldots, a_{k-1}$ are (symbols for) nonnegative integers less than $b$, $a_{k-1} \neq 0$, and

$$n = \sum_{i=0}^{k-1} a_i b^i$$

Notice: *The base $b$ expansion of a positive integer $n$ is a string over the alphabet $\{x \in \mathbb{N} \mid x < b\}$ whose leftmost character is nonzero.*

| Base $b$ | Collection of possible coefficients in base $b$ expansion of a positive integer |
|---|---|
| Binary ($b = 2$) | $\{0, 1\}$ |
| Ternary ($b = 3$) | $\{0, 1, 2\}$ |
| Octal ($b = 8$) | $\{0, 1, 2, 3, 4, 5, 6, 7\}$ |
| Decimal ($b = 10$) | $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ |
| Hexadecimal ($b = 16$) | $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$<br>letter coefficient symbols represent numerical values $(A)_{16} = (10)_{10}$<br>$(B)_{16} = (11)_{10}$ $(C)_{16} = (12)_{10}$ $(D)_{16} = (13)_{10}$ $(E)_{16} = (14)_{10}$ $(F)_{16} = (15)_{10}$ |

# Base expansion examples

*Examples*:

$(1401)_2$

$(1401)_{10}$

$(1401)_{16}$

CC BY-NC-SA 2.0 Version October 21, 2026 (3)

# Fixed width definition

**Definition** For $b$ an integer greater than 1, $w$ a positive integer, and $n$ a nonnegative integer _____, the **base** $b$ **fixed-width** $w$ **expansion of** $n$ is

$$(a_{w-1} \cdots a_1 a_0)_{b,w}$$

where $a_0, a_1, \ldots, a_{w-1}$ are nonnegative integers less than $b$ and

$$n = \sum_{i=0}^{w-1} a_i b^i$$

# Representing zero

**Representing** 0: Which of the representations so far allow us to represent 0?

# Base expansion review

Find and fix any and all mistakes with the following:

(a) $(1)_2 = (1)_8$

(b) $(142)_{10} = (142)_{16}$

(c) $(20)_{10} = (10100)_2$

(d) $(35)_8 = (1D)_{16}$

CC BY-NC-SA 2.0 Version October 21, 2026 (4)

# Algorithm definition

**New!** An algorithm is a finite sequence of precise instructions for solving a problem.

Algorithms can be expressed in English or in more formalized descriptions like pseudocode or fully executable programs.

Sometimes, we can define algorithms whose output matches the rule for a function we already care about. Consider the (integer) logarithm function

$$logb : \{b \in \mathbb{Z} \mid b > 1\} \times \mathbb{Z}^+ \quad \rightarrow \quad \mathbb{N}$$

defined by

$$logb(\ (b, n)\ ) = \text{greatest integer } y \text{ so that } b^y \text{ is less than or equal to } n$$

Calculating integer part of base $b$ logarithm

```
1   procedure logb(b,n: positive integers with b>1)
2   i := 0
3   while n > b-1
4     i := i+1
5     n := n div b
6   return i {i holds the integer part of the base b logarithm of n}
```

Trace this algorithm with inputs $b = 3$ and $n = 17$

| | $b$ | $n$ | $i$ | $n > b - 1$? |
|---|---|---|---|---|
| Initial value | 3 | 17 | | |
| After 1 iteration | | | | |
| After 2 iterations | | | | |
| After 3 iterations | | | | |
| | | | | |

Compare: does the output match the rule for the (integer) logarithm function?

CC BY-NC-SA 2.0 Version October 21, 2026 (5)

# Base expansion algorithms

### Two algorithms for constructing base $b$ expansion from decimal representation

**Most significant first**: Start with left-most coefficient of expansion (highest value)

*Informally*: Build up to the value we need to represent in "greedy" approach, using units determined by base.

Calculating base $b$ expansion, from left

```
1  procedure basebmost(n,b: positive integers with b>1)
2  v := n
3  k := 1+ output of logb algorithm with inputs b and n
4  for i := 1 to k
5     a_{k-i} := 0
6     while v ≥ b^{k-i}
7        a_{k-i} := a_{k-i} + 1
8        v := v - b^{k-i}
9  return (a_{k-1},...,a_0){(a_{k-1}...a_0)_b is the base b expansion of n}
```

CC BY-NC-SA 2.0 Version October 21, 2026 (6)

**Least significant first**: Start with right-most coefficient of expansion (lowest value)

Idea: (when $k > 1$)

$$\begin{aligned} n &= a_{k-1}b^{k-1} + \cdots + a_1 b + a_0 \\ &= b(a_{k-1}b^{k-2} + \cdots + a_1) + a_0 \end{aligned}$$

so $a_0 = n$ **mod** $b$ and $a_{k-1}b^{k-2} + \cdots + a_1 = n$ **div** $b$.

Calculating base $b$ expansion, from right

```
1  procedure basebleast(n,b: positive integers with b>1)
2  k := 0
3  q := n
4  while q ≠ 0
5     a_k := q mod b
6     k := k+1
7     q := q div b
8  return (a_{k-1},...,a_0){(a_{k-1}...a_0)_b is the base b expansion of n}
```

## Base conversion algorithm

Practice: write an algorithm for converting from base $b_1$ expansion to base $b_2$ expansion:

## Defining sets

*To define sets:*

To define a set using **roster method**, explicitly list its elements. That is, start with { then list elements of the set separated by commas and close with }.

To define a set using **set builder definition**, either form "The set of all $x$ from the universe $U$ such that $x$ is ..." by writing

$$\{x \in U \mid ...x...\}$$

or form "the collection of all outputs of some operation when the input ranges over the universe $U$" by writing

$$\{...x... \mid x \in U\}$$

We use the symbol $\in$ as "is an element of" to indicate membership in a set.

CC BY-NC-SA 2.0 Version October 21, 2026 

**Example sets**: For each of the following, identify whether it's defined using the roster method or set builder notation and give an example element.

Can we infer the data type of the example element from the notation?

$\{-1, 1\}$

$\{0, 0\}$

$\{-1, 0, 1\}$

$\{(x, x, x) \mid x \in \{-1, 0, 1\}\}$

$\{\}$

$\{x \in \mathbb{Z} \mid x \geq 0\}$

$\{x \in \mathbb{Z} \mid x > 0\}$

$\{\smile, \sun\}$

$\{\texttt{A}, \texttt{C}, \texttt{U}, \texttt{G}\}$

$\{\texttt{AUG}, \texttt{UAG}, \texttt{UGA}, \texttt{UAA}\}$

# Rna motivation

RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set $B = \{\texttt{A}, \texttt{C}, \texttt{U}, \texttt{G}\}$. Strands are ordered nonempty finite sequences of bases.

Formally, to define the set of all RNA strands, we need more than roster method or set builder descriptions.

# Set recursive examples

**Definition** The set of nonnegative integers $\mathbb{N}$ is defined (recursively) by:

Basis Step:
Recursive Step:

Examples:

**Definition** The set of all integers $\mathbb{Z}$ is defined (recursively) by:

Basis Step:
Recursive Step:

Examples:

**Definition** The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

Examples:

**Definition** The set of bitstrings (strings of 0s and 1s) is defined (recursively) by:

Basis Step:
Recursive Step:

*Notation:* We call the set of bitstrings $\{0,1\}^*$ and we say this is the set of all strings over $\{0,1\}$.

Examples:

CC BY-NC-SA 2.0 Version October 21, 2026 (10)

# Defining functions more examples

Let's practice with functions related to some of our applications so far.

Recall: We model the collection of user ratings of the four movies Superman, Wicked, KPop Demon Hunters, A Minecraft Movie as the set $\{-1, 0, 1\}^4$ . One function that compares pairs of ratings is

$$d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \to \mathbb{R}$$

given by

$$d_0(\ (\ (x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)\ )\ ) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

Notice: any ordered pair of ratings is an okay input to $d_0$.

Notice: there are (at most)

$$(3 \cdot 3 \cdot 3 \cdot 3) \cdot (3 \cdot 3 \cdot 3 \cdot 3) = 3^8 = 6561$$

many pairs of ratings. There are therefore lots and lots of real numbers that are not the output of $d_0$.

Recall: RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set $B = \{\mathtt{A}, \mathtt{C}, \mathtt{U}, \mathtt{G}\}$. The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

**Pro-tip**: informal definitions sometime use $\cdots$ to indicate "continue the pattern". Often, to make this pattern precise we use recursive definitions.

CC BY-NC-SA 2.0 Version October 21, 2026 (11)

| Name | Domain | Codomain | Rule | Example |
|---|---|---|---|---|
| *rnalen* | $S$ | $\mathbb{Z}^+$ | Basis Step:<br>If $b \in B$ then $rnalen(b) = 1$<br>Recursive Step:<br>If $s \in S$ and $b \in B$, then<br>$rnalen(sb) = 1 + rnalen(s)$ | $rnalen(\texttt{AC}) \overset{\text{rec step}}{=} 1 + rnalen(\texttt{A})$<br>$\overset{\text{basis step}}{=} 1 + 1 = 2$ |
| *basecount* | $S \times B$ | $\mathbb{N}$ | Basis Step:<br>If $b_1 \in B, b_2 \in B$ then<br>$basecount(\ (b_1, b_2)\ ) =$<br>$\begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$<br>Recursive Step:<br>If $s \in S, b_1 \in B, b_2 \in B$<br>$basecount(\ (sb_1, b_2)\ ) =$<br>$\begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$ | $basecount(\ (\texttt{ACU}, \texttt{C})\ ) =$ |
| "2 to the power of" | $\mathbb{N}$ | $\mathbb{N}$ | Basis Step:<br>$2^0 = 1$<br>Recursive Step:<br>If $n \in \mathbb{N}, 2^{n+1} =$ | |
| "$b$ to the power of $i$" | $\mathbb{Z}^+ \times \mathbb{N}$ | $\mathbb{N}$ | Basis Step:<br>$b^0 = 1$<br>Recursive Step:<br>If $i \in \mathbb{N}, b^{i+1} = b \cdot b^i$ | |

$2^0 = 1$ $2^1 = 2$ $2^2 = 4$ $2^3 = 8$ $2^4 = 16$ $2^5 = 32$ $2^6 = 64$ $2^7 = 128$ $2^8 = 256$ $2^9 = 512$ $2^{10} = 1024$

CC BY-NC-SA 2.0 Version October 21, 2026 (12)

# Division algorithm

**Integer division and remainders** (aka The Division Algorithm) Let $n$ be an integer and $d$ a positive integer. There are unique integers $q$ and $r$, with $0 \leq r < d$, such that $n = dq + r$. In this case, $d$ is called the divisor, $n$ is called the dividend, $q$ is called the quotient, and $r$ is called the remainder.

Because these numbers are guaranteed to exist, the following functions are well-defined:

**div** $: \mathbb{Z} \times \mathbb{Z}^+ \to \mathbb{Z}$ given by **div** ( $(n, d)$ ) is the quotient when $n$ is the dividend and $d$ is the divisor.

**mod** $: \mathbb{Z} \times \mathbb{Z}^+ \to \mathbb{Z}$ given by **mod** ( $(n, d)$ ) is the remainder when $n$ is the dividend and $d$ is the divisor.

Because these functions are so important, we sometimes use the notation $n$ **div** $d =$ **div** ( $(n, d)$ ) and $n$ **mod** $d =$ **mod** ( $(n, d)$ ).

**Pro-tip**: The functions **div** and **mod** are similar to (but not exactly the same as) the operators / and % in Java and python.

*Example calculations*:

20 **div** 4

20 **mod** 4

20 **div** 3

20 **mod** 3

$-20$ **div** 3

$-20$ **mod** 3

CC BY-NC-SA 2.0 Version October 21, 2026 (13)